Lecture 1b: Feature Vectors & Data Modalities

Omkar Bait (CosmicAI Fellow)
NRAO, Charlottesville

Artificial Intelligence Institute for Cosmic Origins
Funded by the U.S. National Science Foundation and the Simons Foundation

Step 1 — Frame the Problem

Before touching data or models, write down:

What am I predicting? Is the target a class (e.g. spiral vs. elliptical) or a number (e.g. redshift z)?

How many classes? Binary (star / not-star) or multi-class (E, S0, Sa, Sb, Sc, Irr)?

What does 'success' look like? Accuracy? RMSE on z? A photometric-redshift bias $|\Delta z| < 0.05$?

What inputs are available? Photometric magnitudes, image cutouts, spectra, redshift catalogs?

Astro example: "Given ugrizy magnitudes from LSST, predict the spectroscopic redshift z." → *regression task, continuous target.*

Step 2 — Features (X) and Targets (y)

Each example is an n-dimensional feature vector:

$$\mathbf{X} = (\ x_1,\ \ x_2,\ \ x_3,\ \ \ldots\ ,\ \ x_n\ )$$

The model learns a mapping $f : X \rightarrow y$, where y is the ***target***.

One SDSS galaxy:

```
X = (u, g, r, i, z) =
      (19.4, 18.1, 17.4,
17.0, 16.8)
```

```
y = z_spec = 0.124
```

Vocabulary check:

Name	Symbol	Also called	Example
Features	**x**	independent variables, predictors, inputs	ugrizy magnitudes
Target	**y**	dependent variable, label, response	redshift z, galaxy type

Step 3 — Statistical Data Scales

Not all features are created equal. The data type determines what you can do with it.

Categorical (Qualitative)

Nominal — no inherent order.
spiral / elliptical / irregular
galaxy ID, filter band (u,g,r,...)

Ordinal — ranked, but spacing not meaningful.
image quality: poor < OK < excellent
Hubble stage: E → S0 → Sa → Sb → Sc

Continuous (Quantitative)

Interval — differences are meaningful.
apparent magnitude in r-band
temperature (°C), RA / Dec coordinates

Ratio — true zero → ratios are meaningful.
flux, luminosity, stellar mass M☆
redshift z, distance in Mpc

Why it matters: categorical features cannot be fed directly into a tree or neural net — they must be *encoded* (Lecture 2). Ordinal vs nominal changes which encoding is correct.

Worked Example — Data Types

WORKED EXAMPLE

For each feature, decide if it is Nominal, Ordinal, Interval, or Ratio. Try it for 1 minute, then advance for the answer.

Feature	Example value	Your answer
galaxy_type	"spiral", "elliptical", "irregular"	________________
seeing_quality	"poor" < "OK" < "excellent"	________________
apparent magnitude (r-band)	18.5 mag	________________
RA coordinate	245.3 deg	________________
flux in r-band	1.5×10^{-7} erg / s / cm^2	________________
redshift z	0.12	________________

Worked Example — Solution

ANSWER

Feature	Type	Why
galaxy_type	**Nominal**	*no inherent order*
seeing_quality	**Ordinal**	*ranked but spacing not meaningful*
apparent magnitude	**Interval**	*differences meaningful; no true zero (log scale)*
RA coordinate	**Interval**	*0° is arbitrary (the vernal equinox)*
flux (r-band)	**Ratio**	*zero flux IS true zero; ratios meaningful*
redshift z	**Ratio**	*z = 0 means stationary; z1/z2 is meaningful*

Why this matters in Lecture 2:
Nominal → one-hot encode. **Ordinal** → label-encode preserving the order. **Interval / Ratio** → use as-is, optionally scale (Standard or MinMax).

Quick Check 1

QUIZ

Which of these is BEST treated as an ORDINAL feature?

A

Galaxy ID number
(e.g. 587725073921...)

B

Hubble morphological stage
E → S0 → Sa → Sb → Sc

C

Spectroscopic redshift
$z = 0.12$

D

Galaxy color (label)
"blue" / "red" / "green"

Pick A, B, C, or D — then advance for the answer.

Quick Check 1 — Answer

ANSWER

A Galaxy ID number

Nominal — IDs have no meaningful order

B **Hubble morphological stage** ✓

Ranked: E < S0 < Sa < Sb < Sc

C Spectroscopic redshift

Ratio — continuous numeric value

D Galaxy color (label)

Nominal — labels have no order

Why B: Hubble stage has an inherent rank — but the spacing between Sa, Sb, Sc isn't a uniform numeric quantity. That's the textbook definition of ***ordinal.***

Step 4 — Exploratory Data Analysis

EDA is the first conversation with your data — before any model.

1. Basic shape

How many examples (rows)? How many features (cols)? Any missing values? Are types numeric or strings?

2. Distributions

Histogram & box plot per feature. Skewed? Bimodal? Outliers? Same scale across features?

3. Relationships

Correlation matrix, pair plots, scatter plots. Are some features redundant? Do features track the target?

4. Class & sample balance

If classification: is one class 95% of the data? If regression: is the target uniformly sampled, or biased to bright/nearby?

Rule of thumb: skip EDA → discover your data bugs in the *model results*.

Worked Example — Frame It

WORKED EXAMPLE

Scenario

You receive an SDSS catalog of **50 000** galaxies. Each row has:

`u, g, r, i, z magnitudes` *(continuous numeric)*

a measured spectroscopic redshift **`z_spec`**

`galaxy_id`

Your goal: predict z for ***new*** galaxies that have only photometry (no spectroscopy).

Write down on paper:

1. Classification or regression?
2. What is X?
3. What is y?
4. What's a good success metric?

Pause and try — then advance.

Worked Example — Solution

ANSWER

1. Classification or regression?

Regression

z is a continuous real number (no discrete classes).

2. What is X?

`X = (u, g, r, i, z)`

5 photometric magnitudes — optionally with derived colors (u-g, g-r, r-i).

3. What is y?

`y = z_spec`

The single column we want to predict, for each galaxy.

4. Success metric?

RMSE + bias + scatter

Standard photo-z trio: $\sigma(\Delta z)$, $\langle \Delta z \rangle$, and outlier fraction.

You just framed the photo-z problem — the exact case study you'll solve in **Lecture 4!**

EDA in Practice — Distributions

What to look at per feature:

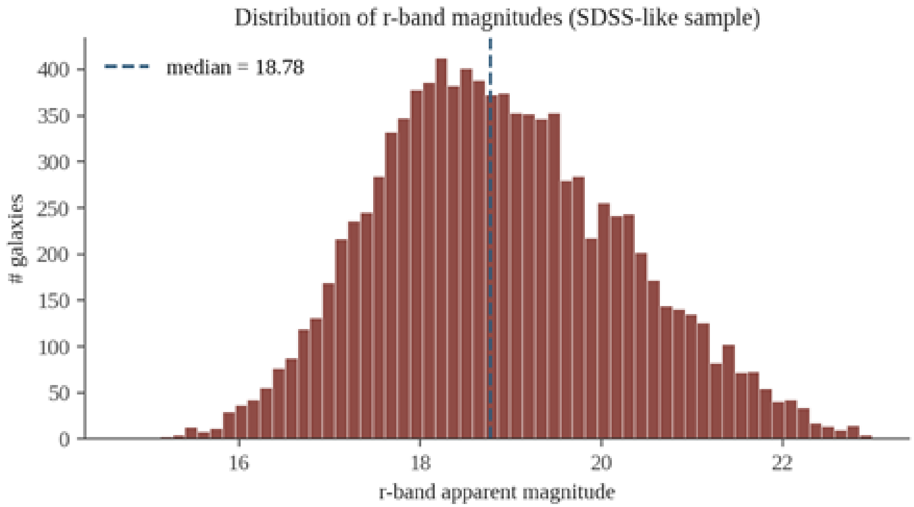
Center: mean, median.

Spread: std-dev, IQR, range.

Shape: skew, heavy tails, bimodality.

Outliers: Z-score > 3 or > 1.5·IQR (Lec 2).

Scale: are different features on wildly different ranges?



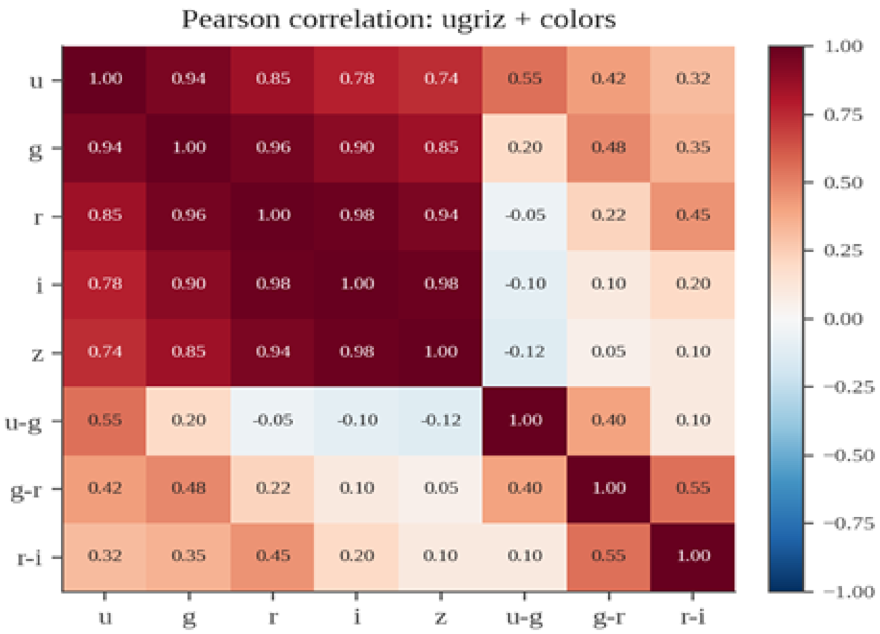
Astro takeaway: magnitudes are usually skewed (a faint tail). Flux / luminosity are even more skewed (log-normal). Often we work with `log(flux)` instead of `flux` to get a friendlier shape for the model.

EDA in Practice — Correlations

Pearson correlation $r \in [-1, +1]$:

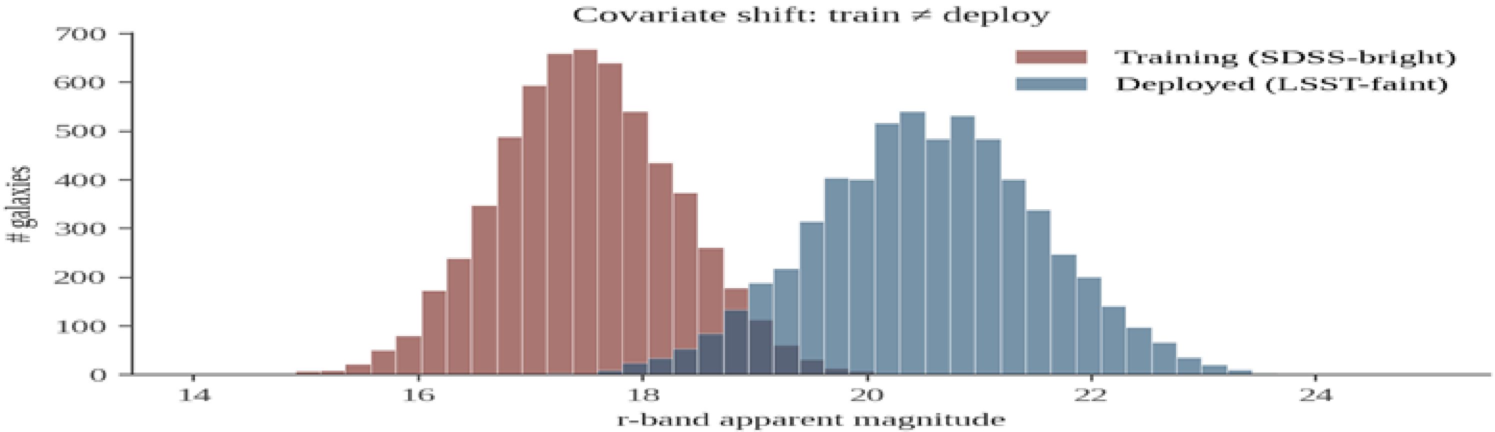
+1	perfect positive linear
0	no linear relationship
−1	perfect negative linear

Highly correlated features carry redundant information — we'll prune or combine in *Lec 2* (feature selection, PCA).



Why the bands all look red: ugriz magnitudes of one galaxy move together — brighter in r usually means brighter in i, g, z. Colors (u–g, g–r) decorrelate this and carry the physics.

EDA Gotcha — Distribution Shift



What you see:
Your training set is *brighter*, your deployment data is *fainter*. The model is being asked to extrapolate.

Why it matters:
test-set accuracy was a *lie* — it measured the wrong distribution. Photo-z catalogs notoriously break on faint galaxies for exactly this reason.

Detect it early: overlay histograms of every feature, train vs deploy. If they don't overlap, you're in trouble.

Dimensionality — How Many Features?

Each feature is one axis. Adding a feature = one more axis.

d = 1

r-band magnitude

A line of galaxies on one axis. Easy to plot, easy to learn — but very little information.

d = 5

ugriz magnitudes

A point in a 5-D color/brightness space. This is the regime of classic photometric redshift.

d = 10^4+

raw galaxy image

A 64×64 cutout is 4 096 features. Now you need CNNs, lots of data, and clever architectures.

Intuition: more features can help (more information per example) — **but only if your dataset is big enough to support them.**

The Curse of Dimensionality

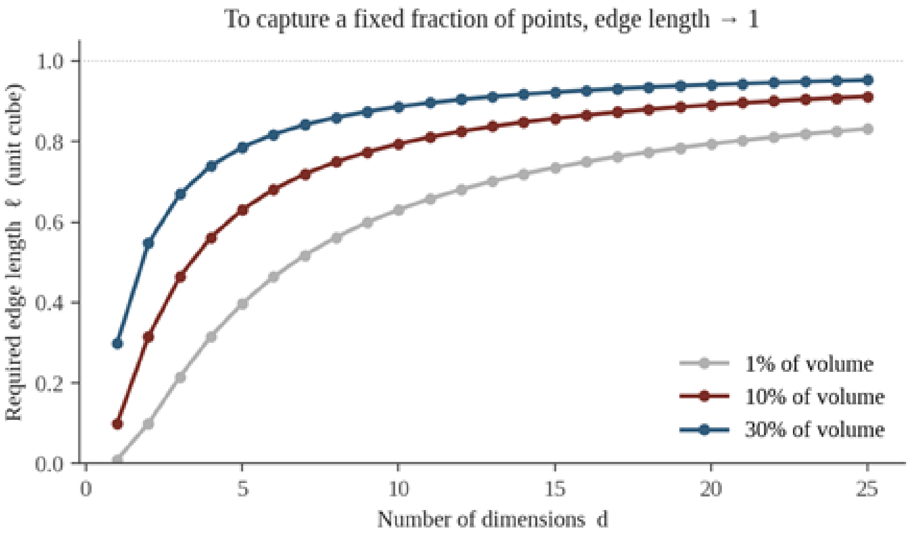
To capture a fraction p of points in a uniform d-dimensional unit cube, you need a sub-cube of edge length:

$$\ell(d) = p^{(1/d)}$$

Worked example (p = 10%):

```
d = 1  →  ℓ = 0.10  (10% of axis)
d = 5  →  ℓ ≈ 0.63  (63% of each axis)
d = 10 →  ℓ ≈ 0.79  (almost the whole cube!)
```

→ In high-D, **all points are far apart.** Nearest-neighbour intuitions stop working.



Practical implication: the more features you add, the more data (and the more clever models) you need to fill that space. Otherwise you'll overfit.

Quick Check 2

QUIZ

You're predicting a value from $N = 1000$ examples. As you add MORE features (raise d), what happens to your model's ability to learn?

A Improves linearly
— more info is always better

B Stays the same
— modern algorithms handle any d

C Degrades roughly exponentially
in d (curse of dimensionality)

D Only matters if features are
improperly scaled

Pick A, B, C, or D — then advance.

Quick Check 2 — Answer

ANSWER

A Improves linearly

Counter-intuitive but false — for fixed N, more features ⇒ sparser space.

B Stays the same

Even modern models benefit from feature pruning / engineering.

C **Degrades exponentially in d** ✓

Curse of dimensionality — see ℓ = p^(1/d).

D Only matters if features unscaled

Scaling helps but doesn't fix the geometric curse.

Why C: To cover a fraction p of the unit cube in d-D, the side length must be `ℓ = p^(1/d).` At $d = 10$, $\ell \approx 0.79$ just to capture 10%. With N fixed, more features → sparser space → poorer learning.

Code — First 5 Minutes With a Dataset

In any new project, run these commands before anything else.

```
import pandas as pd
from sklearn.datasets import fetch_openml

df = pd.read_csv("sdss_photometry.csv")
df.shape          # (n_examples, n_features)
df.dtypes         # numeric vs string vs categorical
df.head()         # first 5 rows — sanity check
df.describe()     # count, mean, std, min, quartiles,
max
df.isna().sum()   # missing values per column

df.hist(bins=50, figsize=(12, 8))  # distributions
df.corr()                           # correlations
```

After running, ask yourself:

- How big is the dataset?
- Any features mis-typed?
- Any column dominated by NaN?
- Wildly different feature scales?
- Skewed distributions to transform?
- Strong correlations?

Recap — The ML Pipeline Starts Here

1	**Frame the problem**	What am I predicting? Classification or regression? What does success mean?
2	**Identify X and y**	Write down the n features (X) and the target (y) explicitly.
3	**Classify your data scales**	Nominal, ordinal, interval, ratio — drives encoding & scaling later.
4	**Do EDA**	Distributions, correlations, class balance, distribution shifts.
5	**Sanity-check dimensionality**	More features → need more data. Beware the curse of dimensionality.

Next → Lecture 2: Data Preparation — imputation, scaling, encoding, dimensionality reduction.